ICPC Reference Book

Arteaga Adán

Algorithms Sheet

Competitive Programming Reference

C++ version

GitHub-ready LaTeX Reference

Índice

1 Base C++ y complejidad

1.1 Fast I/O

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

1.2 Precisión decimal

```
cout << fixed << setprecision(10) << ans << '\n';
```

1.3 Límites de datos

- `int`: $\approx 2 \cdot 10^9$
- `long long`: $\approx 9 \cdot 10^{18}$
- `__int128`: $\approx 10^{38}$, solo GCC.
- `float`: 7 dígitos de precisión.
- `double`: 15-17 dígitos de precisión.
- `long double`: 18-19 dígitos de precisión.

1.4 Truco de memoria

```
// 256 MB aprox:
int a[64 * 1024 * 1024];
```

1.5 Complejidades comunes

- $O(1)$: acceso a índice de arreglo.
- $O(\log n)$: binary search, `std::set`.
- $O(\sqrt{n})$: primalidad, divisores.
- $O(n)$: un recorrido lineal.
- $O(n \log n)$: sorting, heap.
- $O(n^2)$: doble loop, algunas DP.
- $O(2^n)$: subconjuntos, bitmasking.
- $O(n!)$: permutaciones.

1.6 Restricciones estándar

- 10^8 operaciones por segundo, aproximado.
- $n = 10^5 \to O(n \log n)$ u $O(n)$.
- $n = 500 \to O(n^3)$.
- $n = 20 \to O(2^n \cdot n)$.

1.7 Complejidad STL

vector

- `push_back()`, `pop_back()`, `[]` $\to O(1)$.
- `erase(iterator)` $\to O(n)$.

set / map / multiset

- `insert()`, `erase()`, `find()` $\to O(\log n)$.

unordered_map / unordered_set

- `insert()`, `find()` $\to$ promedio $O(1)$, peor caso $O(n)$.

priority_queue

- `push()`, `pop()` $\to O(\log n)$.
- `top()` $\to O(1)$.

1.8 Algoritmos importantes

Math & Number Theory

- **GCD Euclides**: $O(\log(\text{mín}(a, b)))$.
- **Binary Exponentiation**: $O(\log exp)$.
- **Sieve of Eratosthenes**: $O(n \log \log n)$.
- **Prime Factorization**: $O(\sqrt{n})$.

Graphs

- **DFS / BFS**: $O(V + E)$.
- **Dijkstra**: $O(E \log V)$.
- **Floyd-Warshall**: $O(V^3)$.
- **DSU Union-Find**: $O(\alpha(n))$.
- **Segment Tree Query/Update**: $O(\log n)$.

Técnicas generales

- **Two Pointers / Sliding Window**: $O(n)$.
- **Lower / Upper Bound**: $O(\log n)$.
- **Prefix Sum Query**: $O(1)$.
- **Iterating Submasks**: $O(3^n)$.
- **Coordinate Compression**: $O(n \log n)$.

2 Matemáticas, modular y teoría de números

2.1 GCD y LCM

Built-in GCC:

```
// Requiere #include <algorithm>
int g = __gcd(a, b);
```

Euclides manual recursivo:

```
long long gcd(long long a, long long b) {
    return b ? gcd(b, a % b) : a;
}
```

LCM seguro:

```
long long lcm(long long a, long long b) {
    if (a == 0 || b == 0) return 0;
    return (a / __gcd(a, b)) * b;
}
```

Propiedades:

- $gcd(a, b) \cdot lcm(a, b) = |a \cdot b|$.
- $gcd(a, b) = gcd(b, a \bmod b)$.
- $gcd(a, b, c) = gcd(a, gcd(b, c))$.

2.2 Sumas notables

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

2.3 Propiedades de módulo

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

$$(a - b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$$

$$(a \cdot b) \bmod m = ((a \bmod m)(b \bmod m)) \bmod m$$

Negativos:

```
(a % MOD + MOD) % MOD
```

2.4 Binary Exponentiation

```
typedef long long ll;
const ll MOD = 998244353;

ll binpow(ll a, ll b) {
    ll res = 1;
    a %= MOD;
    while (b) {
        if (b & 1) res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }

    return res;
}
```

2.5 Modular Inverse

```
ll modInverse(ll a) {
    return binpow(a, MOD - 2);
}
```

2.6 Modular Factorials

```
const int MAXN = 1e6 + 5;
ll fact[MAXN];

void factorials() {
    fact[0] = 1;
    for (int i = 1; i < MAXN; i++) {
        fact[i] = fact[i - 1] * i % MOD;
    }
}
```

2.7 Modular Combinations

```
ll nCr(ll n, ll r) {
    if (r > n) return 0;
    return fact[n]
        * modInverse(fact[r]) % MOD
        * modInverse(fact[n - r]) % MOD;
}
```

Fórmulas:

$$C(n, r) = \frac{n!}{r!(n-r)!} \qquad P(n, r) = \frac{n!}{(n-r)!}$$

2.8 Criba clásica + factorización

```
const int MAXN = 1e6 + 5;
vector<bool> isPrime(MAXN, true);
vector<int> primes;

void sieve(int n) {
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i <= n; i++) {
        if (isPrime[i]) {
            primes.push_back(i);
            if (1LL * i * i <= n) {
                for (long long j = 1LL * i * i; j <= n; j += i) {
                    isPrime[j] = false;
                }
            }
        }
    }
}
```

```cpp
vector<int> factorize(int x) {
    vector<int> f;
    for (int p : primes) {
        if (1LL * p * p > x) break;
        while (x % p == 0) {
            f.push_back(p);
            x /= p;
        }
    }
    if (x > 1) f.push_back(x);
    return f;
}
```

2.9 Criba lineal SPF + factorización

```cpp
const int MAXN = 1e6 + 5;
vector<int> spf(MAXN, 0), primes;

void sieve(int n) {
    for (int i = 2; i <= n; i++) {
        if (spf[i] == 0) {
            spf[i] = i;
            primes.push_back(i);
        }

        for (int p : primes) {
            if (p > spf[i] || 1LL * i * p > n) break;
            spf[i * p] = p;
        }
    }
}

vector<int> factorize(int x) {
    vector<int> f;
    while (x > 1) {
        f.push_back(spf[x]);
        x /= spf[x];
    }
    return f;
}
```

2.10 Euler's Totient Function ϕ

Single value $O(\sqrt{n})$:

```cpp
int phi(int n) {
    int res = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0) n /= i;
            res -= res / i;
        }
    }
    if (n > 1) res -= res / n;
    return res;
}
```

Totient sieve $O(n \log \log n)$:

```cpp
vector<int> phi(MAXN);

void sieve_phi_simple(int n) {
    for (int i = 1; i <= n; i++) phi[i] = i;
    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) { // i is prime
            for (int j = i; j <= n; j += i) {
                phi[j] -= phi[j] / i;
            }
        }
    }
}
```

3 Arreglos y técnicas generales

3.1 Prefix Sum 1D

Construcción 1-indexed:

```
pref[0] = 0;
for (int i = 1; i <= n; i++) {
    pref[i] = pref[i - 1] + a[i];
}
```

Query $[l, r]$:

$$\text{sum}(l, r) = pref[r] - pref[l-1]$$

Prefix XOR:

$$\text{xor}(l, r) = pref[r] \oplus pref[l-1]$$

3.2 Prefix Sum 2D

Construcción 1-indexed:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        p[i][j] = a[i][j] + p[i - 1][j]
                + p[i][j - 1] - p[i - 1][j - 1];
    }
}
```

Query rectángulo $[x_1, y_1]$ a $[x_2, y_2]$:

```
int res = p[x2][y2] - p[x1 - 1][y2]
        - p[x2][y1 - 1] + p[x1 - 1][y1 - 1];
```

3.3 Difference Array

Update $[l, r]$ con valor v:

```
d[l] += v;
d[r + 1] -= v;
```

Resultado: el valor final de $a[i]$ es la suma de prefijo de d hasta i.

3.4 Binary Search básica

```
int l = 0, r = n - 1;

while (l <= r) {
    int mid = l + (r - l) / 2;

    if (a[mid] == x) {
        break;
    }

    if (a[mid] < x) l = mid + 1;
    else r = mid - 1;
}
```

3.5 Binary Search on Answer

```
while (l <= r) {
    int mid = l + (r - l) / 2;

    if (check(mid)) {
        ans = mid;
        r = mid - 1;
    } else {
        l = mid + 1;
    }
}
```

3.6 lower_bound y upper_bound

lower_bound: primer elemento $\geq x$.

```
auto it = lower_bound(v.begin(), v.end(), x);
int idx = it - v.begin();
if (it == v.end()) {
    // No existe elemento >= x
}
```

upper_bound: primer elemento $> x$.

```
auto it = upper_bound(v.begin(), v.end(), x);
int idx = it - v.begin();
```

Cantidad de x:

```
int cnt = upper_bound(v.begin(), v.end(), x)
        - lower_bound(v.begin(), v.end(), x);
```

3.7 Maximum Subarray Sum Kadane

```
long long max_so_far = -1e18, current_max = 0;

for (int i = 0; i < n; i++) {
    current_max += a[i];
    if (max_so_far < current_max) max_so_far = current_max;
    if (current_max < 0) current_max = 0;
}
```

3.8 LIS Longest Increasing Subsequence

Código $O(n \log n)$:

```
vector<int> lis;

for (int x : a) {
    auto it = lower_bound(lis.begin(), lis.end(), x);
    if (it == lis.end()) lis.push_back(x);
    else *it = x;
}

int ans = (int)lis.size();
```

Nota: usa `upper_bound` para Longest Non-Decreasing Subsequence.

4 Bit Manipulation

4.1 Operaciones básicas

Set, check, toggle, clear:

```
mask | (1LL << i)     // Set bit i
mask & (1LL << i)     // Check bit i
mask ^ (1LL << i)     // Toggle bit i
mask & ~(1LL << i)    // Clear bit i
```

LSB Least Significant Bit:

```
mask & -mask          // Get LSB isolated
mask & (mask - 1)     // Remove LSB
```

4.2 Built-in functions GCC

```
__builtin_popcountll(n) // Count 1s
__builtin_ctzll(n)      // Count trailing 0s
__builtin_clzll(n)      // Count leading 0s
__builtin_ffsll(n)      // 1-based index of LSB
```

Usa sufijo `ll` para `long long`.

4.3 Técnicas y trucos

All bits set of size n:

```
(1LL << n) - 1
```

Check if power of two:

```
n && !(n & (n - 1))
```

XOR properties:

- $a \oplus a = 0$.
- $a \oplus 0 = a$.
- Si $a \oplus b = c$, entonces $a \oplus c = b$.

4.4 Iterar subconjuntos

Todos los subconjuntos de n elementos:

```
for (int i = 0; i < (1 << n); i++) {
    for (int j = 0; j < n; j++) {
        if (i & (1 << j)) {
            // j is in subset
        }
    }
}
```

4.5 Iterar submasks

```
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
    // sub is a submask of mask
}
// Excludes empty mask 0
```

4.6 Subsets de tamaño K Gosper's Hack

```
int set = (1 << k) - 1;

while (set < (1 << n)) {
    // Process set
    int c = set & -set;
    int r = set + c;
    set = (((r ^ set) >> 2) / c) | r;
}
```

4.7 Fast log2 integer

```
31 - __builtin_clz(n)    // int
63 - __builtin_clzll(n)  // long long
```

5 Strings

5.1 getline()

```
string s;
getline(cin, s);
```

Uso: lee toda una línea incluyendo espacios.

5.2 cin.ignore()

Precaución: si antes usaste lectura con `cin`, queda un salto de línea pendiente.

```
int n;
cin >> n;
cin.ignore();

string s;
getline(cin, s);
```

Uso: evita que `getline` lea una línea vacía.

5.3 stringstream

```
string s = "10 20 30";
stringstream ss(s);

int a, b, c;
ss >> a >> b >> c;
```

Uso: permite separar datos desde un `string` como si fuera `cin`.

5.4 Z-function

Uso: pattern matching, bordes y prefijos. $O(n)$.

```
vector<int> z_function(const string& s) {
    int n = (int)s.size();
    vector<int> z(n);
    int l = 0, r = 0;

    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }

    return z;
}
```

Buscar patrón:

```
vector<int> find_occurrences_z(string text, string pat) {
    string s = pat + "#" + text;
    vector<int> z = z_function(s), ans;
    int m = (int)pat.size();

    for (int i = m + 1; i < (int)s.size(); i++) {
        if (z[i] == m) ans.push_back(i - m - 1);
    }

    return ans;
}
```

5.5 KMP Prefix Function

Uso: pattern matching y bordes. $O(n)$.

```
vector<int> prefix_function(const string& s) {
    int n = (int)s.size();
    vector<int> pi(n);

    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }

    return pi;
}
```

Buscar patrón:

```
vector<int> kmp_search(string text, string pat) {
    string s = pat + "#" + text;
    vector<int> pi = prefix_function(s), ans;
    int m = (int)pat.size();

    for (int i = m + 1; i < (int)s.size(); i++) {
        if (pi[i] == m) ans.push_back(i - 2 * m);
    }

    return ans;
}
```

5.6 Manacher

Uso: palíndromos en $O(n)$.

```
vector<int> manacher_odd(const string& s) {
    int n = (int)s.size();
    vector<int> d1(n);
    int l = 0, r = -1;

    for (int i = 0; i < n; i++) {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
        while (0 <= i - k && i + k < n && s[i - k] == s[i + k]) {
            k++;
        }
        d1[i] = k;
        if (i + k - 1 > r) {
            l = i - k + 1;
            r = i + k - 1;
        }
    }

    return d1;
}

vector<int> manacher_even(const string& s) {
    int n = (int)s.size();
```

```
    vector<int> d2(n);
    int l = 0, r = -1;

    for (int i = 0; i < n; i++) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k]) {
            k++;
        }
        d2[i] = k;
        if (i + k - 1 > r) {
            l = i - k;
            r = i + k - 1;
        }
    }

    return d2;
}
```

Longest palindromic substring length:

```
int longest_palindrome_len(const string& s) {
    auto d1 = manacher_odd(s);
    auto d2 = manacher_even(s);
    int ans = 0;

    for (int x : d1) ans = max(ans, 2 * x - 1);
    for (int x : d2) ans = max(ans, 2 * x);

    return ans;
}
```

6 Grafos básicos

6.1 Adjacency List

```
const int MAXN = 2e5 + 5;
vector<int> adj[MAXN];
```

Agregar arista no dirigida:

```
adj[u].push_back(v);
adj[v].push_back(u);
```

6.2 Propiedades de grafos

Tree edges:

$$E = n - 1$$

Complete graph edges K_n:

$$E = \frac{n(n-1)}{2}$$

Handshaking Lemma:

$$\sum deg(v) = 2 \cdot |E|$$

6.3 DFS Depth First Search

```
bool vis[MAXN];

void dfs(int u) {
    vis[u] = true;
    for (int v : adj[u]) {
        if (!vis[v]) dfs(v);
    }
}
```

Uso: componentes, árboles y recorridos generales.

6.4 BFS Breadth First Search

```
void bfs(int s) {
    queue<int> q;
    q.push(s);
    vis[s] = true;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (!vis[v]) {
                vis[v] = true;
                q.push(v);
            }
        }
    }
}
```

Uso: shortest path en grafos no ponderados.

6.5 BFS Distances

```
int dist[MAXN]; // Inicializar con -1

void bfs_dist(int s) {
    queue<int> q;
    q.push(s);
    dist[s] = 0;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}
```

6.6 Grid Directions 4-way

```
int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};

for (int k = 0; k < 4; k++) {
    int nx = x + dx[k];
    int ny = y + dy[k];
    // Check bounds: 0 <= nx < R, 0 <= ny < C
}
```

6.7 Coloreado de grafos Bipartite Check

Uso: detectar si un grafo no dirigido puede colorearse con 2 colores. $O(V + E)$.

```
vector<int> color(MAXN, -1);

bool bfs_color(int s) {
    queue<int> q;
    q.push(s);
    color[s] = 0;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (color[v] == -1) {
                color[v] = color[u] ^ 1;
                q.push(v);
            } else if (color[v] == color[u]) {
                return false;
            }
        }
    }

    return true;
}

bool is_bipartite(int n) {
    fill(color.begin(), color.begin() + n + 1, -1);
    for (int i = 1; i <= n; i++) {
        if (color[i] == -1 && !bfs_color(i)) return false;
    }
    return true;
}
```

6.8 Detectar ciclo en grafo no dirigido

```
bool has_cycle_undirected(int u, int parent) {
    vis[u] = true;

    for (int v : adj[u]) {
        if (!vis[v]) {
            if (has_cycle_undirected(v, u)) return true;
        } else if (v != parent) {
            return true;
        }
    }

    return false;
}

bool cycle_undirected_all(int n) {
    fill(vis, vis + n + 1, false);
    for (int i = 1; i <= n; i++) {
        if (!vis[i] && has_cycle_undirected(i, -1)) return true;
    }
    return false;
}
```

6.9 Detectar ciclo en grafo dirigido

Estado: 0 = no visitado, 1 = activo, 2 = terminado.

```
int state[MAXN];

bool has_cycle_directed(int u) {
    state[u] = 1;

    for (int v : adj[u]) {
        if (state[v] == 1) return true;
        if (state[v] == 0 && has_cycle_directed(v)) return true;
    }

    state[u] = 2;
    return false;
}

bool cycle_directed_all(int n) {
    fill(state, state + n + 1, 0);
    for (int i = 1; i <= n; i++) {
        if (state[i] == 0 && has_cycle_directed(i)) return true;
    }
    return false;
}
```

7 Grafos avanzados y DSU

7.1 Dijkstra Shortest Path

```
const long long INF = 1e18;
vector<pair<int, int>> adj[MAXN]; // {v, w}
long long dist[MAXN];

void dijkstra(int s) {
    fill(dist, dist + MAXN, INF);
    dist[s] = 0;

    priority_queue<pair<long long, int>,
                   vector<pair<long long, int>>,
                   greater<pair<long long, int>>> pq;
    pq.push({0, s});

    while (!pq.empty()) {
        long long d = pq.top().first;
        int u = pq.top().second;
```

```
        pq.pop();

        if (d > dist[u]) continue;

        for (auto edge : adj[u]) {
            int v = edge.first;
            int w = edge.second;

            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```

7.2 Disjoint Set Union DSU

```
struct DSU {
    vector<int> parent, sz;

    DSU(int n) {
        parent.resize(n + 1);
        sz.assign(n + 1, 1);
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int i) {
        if (parent[i] == i) return i;
        return parent[i] = find(parent[i]);
    }

    void unite(int i, int j) {
        int root_i = find(i);
        int root_j = find(j);

        if (root_i != root_j) {
            if (sz[root_i] < sz[root_j]) swap(root_i, root_j);
            parent[root_j] = root_i;
            sz[root_i] += sz[root_j];
        }
    }
};
```

8 Estructuras de datos avanzadas

8.1 Segment Tree Sum Range

```
const int MAXN = 2e5;
int t[4 * MAXN], a[MAXN];

void build(int v, int tl, int tr) {
    if (tl == tr) {
        t[v] = a[tl];
    } else {
        int tm = (tl + tr) / 2;
        build(2 * v, tl, tm);
        build(2 * v + 1, tm + 1, tr);
        t[v] = t[2 * v] + t[2 * v + 1];
    }
}

int query(int v, int tl, int tr, int l, int r) {
    if (l > r) return 0;
    if (l == tl && r == tr) return t[v];

    int tm = (tl + tr) / 2;
    return query(2 * v, tl, tm, l, min(r, tm))
         + query(2 * v + 1, tm + 1, tr, max(l, tm + 1), r);
}

void update(int v, int tl, int tr, int pos, int new_val) {
    if (tl == tr) {
        t[v] = new_val;
    } else {
        int tm = (tl + tr) / 2;
        if (pos <= tm) update(2 * v, tl, tm, pos, new_val);
        else update(2 * v + 1, tm + 1, tr, pos, new_val);
        t[v] = t[2 * v] + t[2 * v + 1];
    }
}
```

Nota: para max/min, cambia `+` por `max`/`min` y el neutro a `-INF`/`INF`.

8.2 Fenwick Tree BIT

```
int bit[MAXN];

void add(int idx, int val) {
    for (; idx < MAXN; idx += idx & -idx) {
        bit[idx] += val;
    }
}

int query(int idx) {
    int res = 0;
    for (; idx > 0; idx -= idx & -idx) {
        res += bit[idx];
    }
    return res;
}

int queryRange(int l, int r) {
    return query(r) - query(l - 1);
}
```

Truco: el BIT es 1-indexed. Si tus datos son 0-indexed, suma 1 al índice.

8.3 Policy-Based Data Structures Ordered Set

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

typedef tree<int, null_type,
             less<int>, rb_tree_tag,
             tree_order_statistics_node_update> ordered_set;
```

```cpp
/*
1. find_by_order(k): iterator to k-th smallest, 0-indexed.
2. order_of_key(x): number of elements strictly smaller than x.
*/

void example() {
    ordered_set s;
    s.insert(1);
    s.insert(9);

    auto it = s.find_by_order(0); // k-th
    int c = s.order_of_key(5);    // count < x
}
```